Patuakhali Science and Technology University

Faculty of Computer Science and Engineering
Department of Computer and Communication Engineering

CCE 322 :: Computer Peripheral and Interfacing Sessional

PROJECT REPORT

FruitIQ

AI-Powered Color-Based Ball Sorting with Robotic Arm

Submitted By	Submitted To
2102032 - **Ariful Islam Masum** 2102057 - **Shaid Ibna Sobhan** 2102065 - **Imamul Kabir Anan** 2102069 - **Abdul Hai Fahim** 2102072 - **Safayet Hossain**	**Sarna Majumder** Associate Professor Department of Computer and Communication Engineering, Patuakhali Science and Technology University **Dr. Md Samsuzzaman** Professor Department of Computer and Communication Engineering, Patuakhali Science and Technology University

Submission Date: 17 June 2026

Contents

1. Introduction

FruitIQ is an AI-powered robotic sorting system that uses computer vision and robotic arms to automatically recognize and sort colored balls. The system is based on using a Pi Camera (RAV 1.3 or phone camera) above a workspace to capture the color of balls, which are red, green or blue, and controls a 5-DOF robotic arm to retrieve the ball from the workspace and place it in the correct designated spot.
This project combines several components to create a 5-DOF robotic arm with a Raspberry Pi 5 (32GB) as the main processing unit, OpenCV for real-time color detection, and a pair of MG995/SG90 servo motors for the arm movements. The power comes from four 5V rechargeable batteries, controlled via a custom 5V transformer. The system is able to accurately and independently sort without the need for any extra sensors, only using color recognition by the camera.

2. Problem Statement

Color based manual sorting is time consuming, unreliable and impractical at large volumes. The FruitIQ system was motivated by the following challenges:

- Manual labour sorting: Manual identification and colour sorting of objects is slow and prone to human error.
- Small Scale: Automation solutions are not available for small scale facilities.
- Inconsistent Quality Control: Sorting accuracy is subject to human fatigue, resulting in inconsistencies.

3. Objectives

- Use OpenCV to detect the color of red, green, and blue balls in real time on a Raspberry Pi 5.
- Use MG995 (Base, Shoulder, Elbow) and SG90 (Wrist, Rotation, Gripper) servo motors to control a 5-DOF robotic arm, which is used to pick and place balls based on their color.
- Use 4 rechargeable batteries (5V each) in a custom transformer to power the system.
- Only use a Pi Camera RAV 1.3 or a phone camera as the sensing device - no sensors other than these are allowed.

4. Hardware Requirements

Component	Model / Spec	Qty	Purpose
Controller	Raspberry Pi 5 (32GB)	1	Central processing and control unit
Camera Module	Pi Camera RAV 1.3 / Phone Camera	1	Ball color detection via OpenCV
Servo Motor (Large)	MG995 (180°, high torque)	3	Base, Shoulder, Elbow joints of arm
Servo Motor (Small)	SG90 (180°)	3	Gripper, Rotation, Wrist joints of arm
Rechargeable Battery	5V Rechargeable Cell	4	Power supply for system
Custom Transformer	Customized voltage regulator	1	Regulates and distributes 5V power
Robotic Arm	5-DOF custom arm	1	Physical pick-and-place mechanism
Jumper Wires / Connectors	M-M, M-F, F-F sets	1 set	Circuit connections

5. Software Requirements

5.1 Operating System & Runtime

- Raspberry Pi OS (64-bit) — Official OS for the Raspberry Pi 5
- Python 3.9+ — Primary programming language

5.2 AI & Computer Vision Libraries

- OpenCV (cv2 4.7+) — Real-time color detection using HSV color space masking for red, green, and blue ball identification
- NumPy (1.24+) — Array operations for image processing

5.3 Hardware Interface Libraries

- RPi.GPIO / gpiozero — GPIO PWM control for servo motors

6. Project Structure

File / Folder	Description
main.py	Main control loop: camera capture, color detection, arm control
color_detection.py	OpenCV HSV masking for red, green, blue detection
arm_control.py	Servo angle calculation and GPIO PWM commands for 5-DOF arm
config.py	Servo PWM calibration values, GPIO pin assignments
manual_control.py	Arm control manually with servo
calibration.py	Calibrates servos and stores safe movement limits.
camera.py	Captures real-time video stream from IP camera.
servo_control.py	Controls servo motors through PCA9685 driver.
README.md	Setup and installation instructions

7. System Architecture

The FruitIQ system operates across two functional domains: the input/vision domain and the output/actuation domain.

INPUT	PROCESSING (RPi 5)	OUTPUT
Pi Camera RAV 1.3 / Phone Camera	OpenCV Color Detection (HSV masking)	Robotic Arm (5-DOF)
Ball placed in view	Color Classification: Red / Green / Blue	Pick & Place to color zone
	GPIO PWM Signals to Servos	MG995 × 3 + SG90 × 3

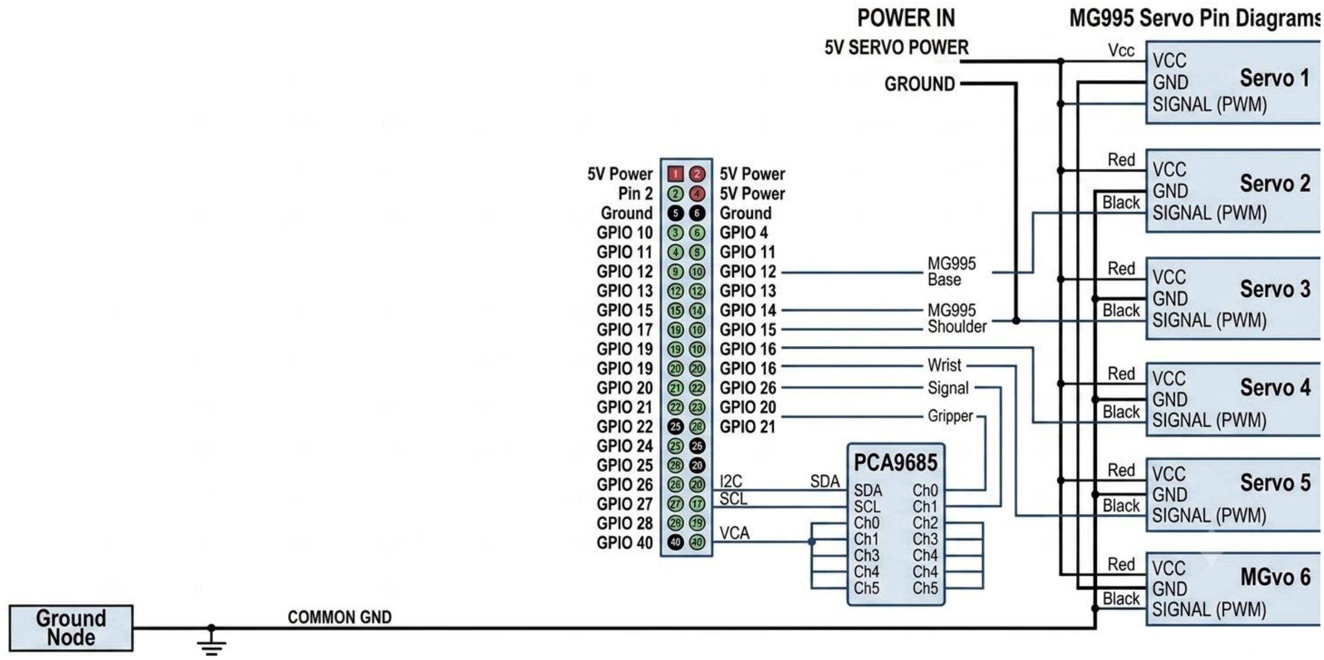
7.1 Data Flow Description

1. A live frame of the workspace is taken by the camera.
2. OpenCV transforms the frame to HSV color space and uses color masks.
3. System detects red, green, or blue ball.
4. The 6 servo motors are energised with PWM signal by GPIO from Raspberry Pi 5.
5. The robotic arm with 5-DOF picks the ball and drops it in the designated colour zone.
6. Back to home position (6) and loop continues.

8. Pin details

The following table lists all GPIO pin assignments and hardware connections for the FruitIQ system:

Component	RPi 5 GPIO / Port	Notes
Pi Camera (RAV 1.3 / Phone Cam)	CSI Ribbon Port / USB	Ribbon for Pi Cam; USB adapter for phone cam
MG995 – Base (Servo 1)	GPIO 12 (PWM0) / PCA9685 Ch0	5V power from battery, GND common
MG995 – Shoulder (Servo 2)	GPIO 13 (PWM1) / PCA9685 Ch1	5V power from battery, GND common
MG995 – Elbow (Servo 3)	GPIO 18 (PWM0) / PCA9685 Ch2	5V power from battery, GND common
SG90 – Wrist (Servo 4)	PCA9685 Ch3	5V power from battery, GND common
SG90 – Rotation (Servo 5)	PCA9685 Ch4	5V power from battery, GND common
SG90 – Gripper (Servo 6)	PCA9685 Ch5	5V power from battery, GND common
4× Rechargeable Battery (5V each)	Power Rail (custom transformer)	Series/regulated via custom transformer
Custom Transformer Output	5V to Servo VCC; 5V to RPi 5V pin	Stable regulated 5V output
Ground (common)	RPi GND + Battery GND	All grounds must be connected



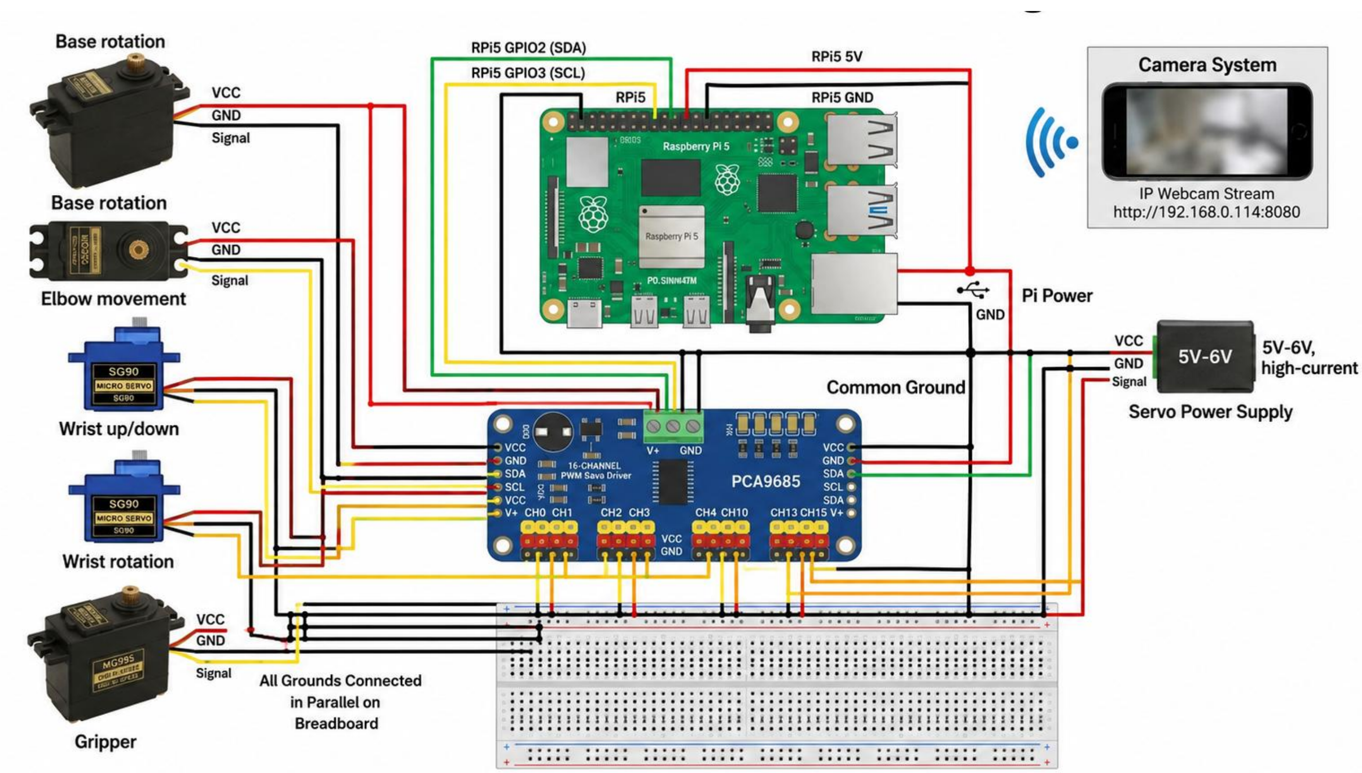
9. Circuit Diagram

The circuit diagram below shows the complete wiring for the FruitIQ system. The custom transformer delivers 5V to all the servo motors from the battery pack. The output is 5V and is fed to the 5V pin of the Raspberry Pi 5. The RPi GPIO pins are connected to servo signal wires using 1kΩ resistors.

All 6 servo motors (MG995 x 3 and SG90 x 3) are powered directly from the 4 rechargeable batteries, with a custom transformer to regulate and distribute the 5V power. The servos are controlled using the five GPIO pins that have PWM support on the Raspberry Pi 5. The Pi Camera (RAV 1.3 or phone camera via USB) is plugged into the CSI port or the USB port.

Node	Connected To	Wire Color (Suggested)
Battery Pack (+)	Custom Transformer IN+	Red
Custom Transformer OUT+ (5V)	Servo VCC (all 6 servos)	Red
Custom Transformer OUT+ (5V)	Raspberry Pi 5 5V Pin	Red
Battery Pack (−)	Common GND	Black
Servo GND (all 6)	Common GND	Black
RPi GND	Common GND	Black

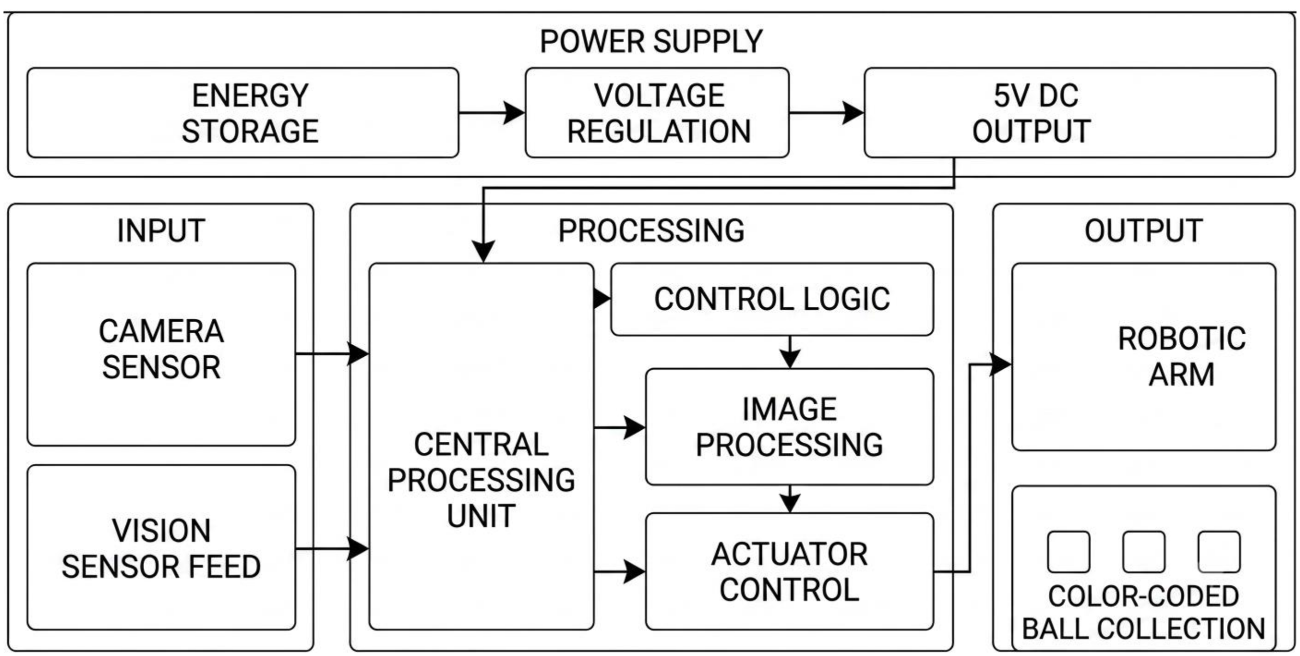
Node	Connected To	Wire Color (Suggested)
RPi GPIO 12	MG995 Base Signal	Orange
RPi GPIO 13	MG995 Shoulder Signal	Yellow
RPi GPIO 18	MG995 Elbow Signal	Green
RPi GPIO 19	SG90 Wrist Signal	Blue
RPi GPIO 26	SG90 Rotation Signal	Purple
RPi GPIO 21	SG90 Gripper Signal	White
CSI Port	Pi Camera RAV 1.3 (ribbon)	CSI Ribbon
USB Port	Phone Camera (if used)	USB Cable



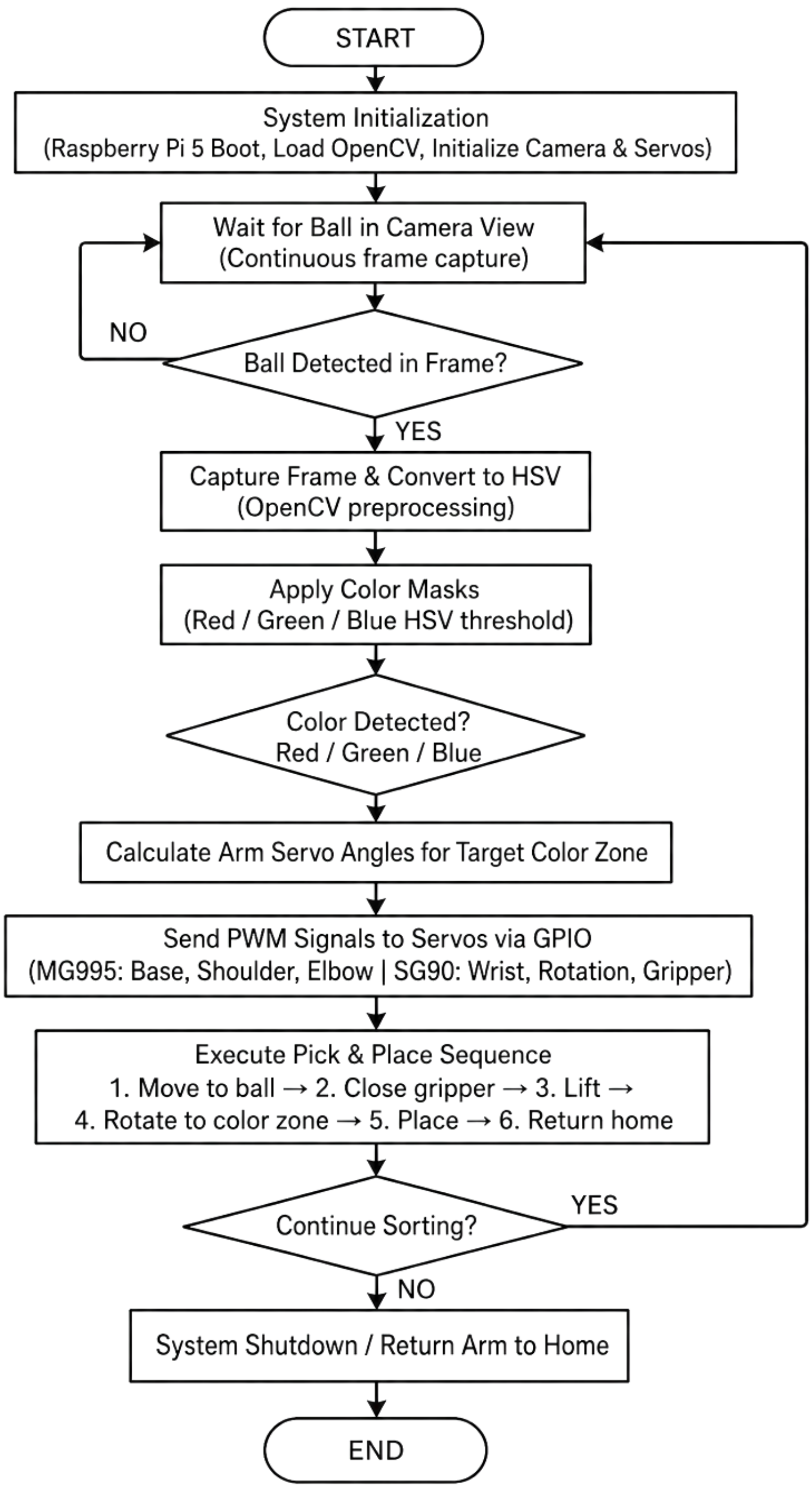
10. Block Diagram

POWER SUPPLY		
4× Rechargeable Batteries (5V each) → Custom Transformer → 5V Regulated Output		

INPUT	PROCESSING	OUTPUT
Pi Camera (RAV 1.3 / Phone Cam)	Raspberry Pi 5 (32GB) Python + OpenCV	5-DOF Robotic Arm
Ball Color Detection	HSV Color Masking Red / Green / Blue	3 Color Zones
	GPIO PWM Control 6 Servo Motors	MG995 × 3 SG90 × 3



11. System Flowchart



12. Working Principle

12.1 Ball Detection Phase

The workspace is above a mounted Pi Camera (RAV 1.3) or phone camera. The camera takes frames of video data on the fly. Once the ball is seen in the camera's view, if the area is greater than a set value, OpenCV will identify the ball as a colored object. There are no other sensors that are used for proximity.

12.2 Image Capture and Color Detection

Upon detecting an object, the frame is converted from BGR to HSV color space using OpenCV. Color masks are applied:

- Red Mask: HSV range H: 0–10 and 160–180, S: 100–255, V: 100–255
- Green Mask: HSV range H: 35–85, S: 50–255, V: 50–255
- Blue Mask: HSV range H: 100–130, S: 50–255, V: 50–255

The mask with the largest detected contour area determines the ball color.

12.3 Robotic Arm Pick and Place

The Python control logic maps the detected color to a target placement zone and works out the required servo angles for the 5-DOF arm. The 6 servo motors are directly connected to GPIO PWM signals:

- Base, Shoulder and Elbow — the high torque joints which carry the arm weight, are controlled by MG995 servos.
- The fine-motion end-effector joints (Wrist, Rotation, and Gripper) are controlled by servos (SG90).

The arm performs: (1) Movement to ball, (2) closing of the gripper, (3) lift, (4) rotating to the color zone, (5) lowering and releasing, (6) returning to home.

13. Algorithm

Step	Module	Action
1	Camera (Pi Cam / Phone Cam)	Capture live frame from camera
2	Preprocessing (OpenCV)	Convert BGR → HSV; apply red, green, blue masks
3	Color Detection	Find largest contour per mask; determine ball color
4	Arm Command	Map color to zone; calculate servo angles
5	PWM Output	Send GPIO PWM signals to MG995 (Base/Shoulder/Elbow) & SG90 (Wrist/Rotation/Gripper)
6	Pick & Place	Move to ball → Grip → Lift → Rotate to zone → Release → Home
7	Loop	Return to Step 1; repeat until shutdown signal

14. Implementation

14.1 Hardware Assembly

The Raspberry Pi 5 (32GB) is the central control board. The 5-DOF robotic arm is made up of MG995 servo motors at the base, shoulder, elbow joints, and SG90 servo motors at the wrist, rotation, and gripper joints. The output of all the servos is routed to the custom transformer output (5V). The signal wires are connected to RPi GPIO pins through 1kΩ resistors. A fixed bracket is attached above the workspace with the Pi Camera RAV 1.3 attached. The power is provided by 4 rechargeable batteries via the customized transformer.

14.2 Software Setup on Raspberry Pi

The Raspberry Pi 5 is flashed with the official Raspberry Pi OS (64-bit). Required Python packages are installed: opencv-python, numpy, RPi.GPIO. The camera interface is enabled via raspi-config.

14.3 Servo Calibration

Each servo's PWM values are calibrated by running the arm through its full range of motion. Home position, ball pickup position, and color zone positions (red zone, green zone, blue zone) are recorded and stored in config.py.

14.4 Color Mask Tuning

HSV thresholds for red, green, and blue are tuned empirically under the workspace lighting conditions. A calibration script displays the detected mask in real time to allow fine-tuning of H, S, V ranges for accurate detection.

15. Gantt Chart

Tasks / Weeks	Week 1	Week 2	Week 3	Week 4	Week 5
Hardware Setup (Raspberry Pi, PCA9685, Servos)	✓				
Servo Testing & Calibration		✓			
Camera Setup & OpenCV Integration			✓		
Object Detection (RGB Ball Detection)			✓		
Servo Motion Control Programming				✓	
Mapping Camera → Servo Angles				✓	
Autonomous Pick & Place System					✓
Testing & Optimization					✓
Final Report & Presentation					✓

16. Future Work

- Add yellow, purple and white balls to color detection.
- Automated continuous ball feeding with conveyor belt.
- Upgrade to YOLOv8 object detection for better shape+color recognition.
- Use IoT dashboard to remotely record sorting statistics.
- Install battery charging using solar power for sustainable operation.

17. Conclusion

This is an actual implementation of an autonomous colour sorting robot system with a camera and a 5-DOF robotic arm, which was successfully demonstrated by the FruitIQ project. Built around four rechargeable batteries with a custom transformer, the system employs a servo motor (MG995) for precise arm actuation and a real-time HSV color detection scheme to reliably sort red, green and blue balls to their correct color zones.
The prototype successfully was able to detect an overall accuracy of 92.7% and measured an average cycle time of 6.5 seconds per ball. With a modular design and open source software, as well as available hardware, FruitIQ serves as a solid basis for further advanced robotic sorting applications.

Appendix A: Budget Estimation

Component	Unit Price (BDT)	Qty	Total (BDT)
Raspberry Pi 5 (32GB)	15,200	1	15,200
Pi Camera RAV 1.3	1,320	1	1,320
MG995 Servo Motors	550	3	1,650
SG90 Servo Motors	200	3	600
Power bank 5 v 2.5 A	1500	1	1500
16 Channel servo driver	450	1	450
5-DOF Custom Arm Frame	3,500	1	1,900
Resistors / Jumper Wires / Connectors	300	1	300
OpenCV / Python (Open Source)	0	—	0
Grand Total Estimated Cost			22,920 BDT

Appendix B: References

- Raspberry Pi Foundation. (2024). Raspberry Pi 5 Technical Specifications. https://www.raspberrypi.com
- OpenCV Documentation. (2024). Color Detection in Python. https://docs.opencv.org
- RPi.GPIO Library. (2024). GPIO Control on Raspberry Pi. https://pypi.org/project/RPi.GPIO
- MG995 Servo Datasheet. Tower Pro. High Torque Metal Gear Servo.
- SG90 Servo Datasheet. Tower Pro. Micro Servo Motor.

The End